BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC CẦN THƠ
TRƯỜNG CÔNG NGHỆ THÔNG TIN & TRUYỀN THÔNG

ĐỒ ÁN

MÔN HỌC NHẬP MÔN TRÍ TUỆ NHÂN TẠO CT223H

Nhóm học phần: M01

Đề tài

BÀI TOÁN QUA SÔNG

Người nông dân, Sói, Dê và Bắp cải

Nhóm sinh viên thực hiện:

1. Nguyễn Thanh Trọng – B2305615 (Trưởng nhóm)
2. Trần Hải Thiên – B2203580
3. Trương Trí Hào – B2203553

Cần Thơ, 03/2026

BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC CẦN THƠ
TRƯỜNG CÔNG NGHỆ THÔNG TIN & TRUYỀN THÔNG

ĐỒ ÁN

MÔN HỌC NHẬP MÔN TRÍ TUỆ NHÂN TẠO CT223H

Nhóm học phần: M01

Đề tài

BÀI TOÁN QUA SÔNG

Người nông dân, Sói, Dê và Bắp cải

Giảng viên hướng dẫn:
TS. Lưu Tiến Đạo

Nhóm sinh viên thực hiện:
1. Nguyễn Thanh Trọng – B2305615
2. Trần Hải Thiên – B2203580
3. Trương Trí Hào – B2203553

Cần Thơ, 03/2026

NHẬN XÉT CỦA GIẢNG VIÊN

Cần Thơ, ngày tháng năm 2026

(Ký và ghi rõ họ tên)

Mục lục

Phân công công việc **5**

1 Mô tả bài toán **5**

1.1 Giới thiệu bài toán qua sông 5

1.2 Phát biểu bài toán 5

1.3 Các thành phần của bài toán tìm kiếm 6

1.3.1 Không gian trạng thái 6

1.3.2 Trạng thái đầu và trạng thái đích 6

1.3.3 Tập hành động 7

1.3.4 Điều kiện hợp lệ 7

1.3.5 Mục tiêu đồ án 7

2 Phân tích bài toán **8**

2.1 Đặc trưng của không gian trạng thái 8

2.2 Biểu diễn trạng thái trong chương trình 8

2.3 Vai trò của ràng buộc 9

2.4 Phân tích heuristic được sử dụng 9

2.4.1 Heuristic `remaining-items` 9

2.4.2 Heuristic `remaining-trips` 10

2.5 Lý do chọn bốn thuật toán 10

3 Cơ sở lý thuyết **10**

3.1 BFS – Breadth-First Search 10

3.2 DFS – Depth-First Search 11

3.3 A* Search 12

3.4 IDA* – Iterative Deepening A* 13

3.5 So sánh lý thuyết giữa bốn thuật toán 13

4 Thiết kế và cài đặt **14**

4.1 Công nghệ sử dụng 14

4.2 Kiến trúc chương trình 14

4.3 Cấu trúc dữ liệu và cách lưu vết tiến trình 15

4.4 Kiểm tra ràng buộc trong mã nguồn 15

4.5 Cài đặt từng nhóm chức năng 16

4.5.1 BFS và DFS 16

4.5.2 A* 16

4.5.3 IDA* 16

4.6 Thiết kế giao diện trực quan 16

5 Kết quả và đánh giá **17**

5.1 Kiểm tra khả năng hoạt động của hệ thống 17

5.2 Lời giải chuẩn của bài toán 17

5.3 Đánh giá theo từng thuật toán 18

5.4 So sánh tìm kiếm mù và tìm kiếm có thông tin 19

5.5 Giá trị của giao diện trực quan 19

5.6 Ưu điểm và hạn chế 19

Phần kết luận **20**

Kết luận 20

Hướng phát triển 21

Tài liệu tham khảo **22**

PHÂN CÔNG CÔNG VIỆC

STT	Họ và tên	MSSV	Công việc
1	Nguyễn Thanh Trọng	B2305615	Phân tích đề tài, mô hình hóa không gian trạng thái, cài đặt BFS và DFS, tổng hợp nội dung báo cáo
2	Trần Hải Thiên	B2203580	Cài đặt A* và IDA*, xây dựng heuristic, kiểm thử quá trình tìm kiếm và đánh giá kết quả
3	Trương Trí Hào	B2203553	Thiết kế giao diện React, trực quan hóa cây trạng thái, chỉnh sửa báo cáo và chuẩn bị phần trình bày

1 Mô tả bài toán

1.1 Giới thiệu bài toán qua sông

Bài toán qua sông là một bài toán kinh điển trong lĩnh vực trí tuệ nhân tạo. Mặc dù số lượng đối tượng ít và luật chơi đơn giản, bài toán vẫn phản ánh rất rõ quá trình mô hình hóa một vấn đề thực tế thành bài toán tìm kiếm. Người học có thể nhìn thấy trực tiếp sự khác biệt giữa việc sinh trạng thái, kiểm tra ràng buộc, chọn nút mở rộng và đánh giá chất lượng lời giải.

Trong đề tài này, bốn thực thể tham gia gồm người nông dân, sói, dê và bắp cải. Mục tiêu là đưa toàn bộ từ bờ trái sang bờ phải bằng một chiếc thuyền nhỏ. Thuyền chỉ đủ chỗ cho người nông dân và tối đa một đối tượng khác. Nếu người nông dân vắng mặt thì sói sẽ ăn dê, còn dê sẽ ăn bắp cải. Vì vậy, lời giải không chỉ cần đưa tất cả qua sông mà còn phải bảo đảm mọi trạng thái trung gian đều an toàn.

Đây là một ví dụ rất phù hợp cho học phần CT223H vì:

- Bài toán có kích thước nhỏ, dễ theo dõi.
- Ràng buộc rõ ràng, thuận tiện cho việc kiểm tra hợp lệ.
- Có thể dùng để so sánh trực tiếp giữa tìm kiếm mù và tìm kiếm có thông tin.
- Phù hợp cho trực quan hóa bằng ứng dụng web.

1.2 Phát biểu bài toán

Người nông dân cần đưa sói, dê và bắp cải từ bờ trái sang bờ phải. Mỗi chuyến đi, thuyền phải có người nông dân điều khiển và chỉ chở thêm được tối đa một đối tượng. Bài toán có các ràng

buộc:

- Nếu sói ở cùng bờ với dê mà không có người nông dân, sói sẽ ăn dê.
- Nếu dê ở cùng bờ với bắp cải mà không có người nông dân, dê sẽ ăn bắp cải.
- Chỉ những trạng thái không vi phạm hai điều kiện trên mới được xem là hợp lệ.

Mục tiêu của bài toán là tìm một chuỗi hành động hợp lệ để đưa toàn bộ bốn thực thể từ bờ trái sang bờ phải một cách an toàn.

1.3 Các thành phần của bài toán tìm kiếm

1.3.1 Không gian trạng thái

Mỗi trạng thái được biểu diễn bởi bộ 4 thành phần:

$$S = (F, W, G, C)$$

Trong đó:

- F: vị trí của người nông dân.
- W: vị trí của sói.
- G: vị trí của dê.
- C: vị trí của bắp cải.

Mỗi thành phần nhận một trong hai giá trị `L` hoặc `R`, tương ứng với bờ trái và bờ phải. Vì có 4 thực thể nên số trạng thái lý thuyết là:

$$2^4 = 16$$

Tuy nhiên nhiều trạng thái trong số này không hợp lệ do vi phạm ràng buộc ăn thịt. Vì vậy bài toán thực tế nhỏ hơn đáng kể so với số trạng thái lý thuyết.

1.3.2 Trạng thái đầu và trạng thái đích

Trạng thái đầu:

$$S_0 = (L, L, L, L)$$

Trạng thái đích:

$$S_{goal} = (R, R, R, R)$$

1.3.3 Tập hành động

Từ một trạng thái bất kỳ, người nông dân có thể thực hiện một trong các hành động:

1. Đi một mình sang bờ đối diện.
2. Chở sói sang bờ đối diện nếu sói cùng bờ với người nông dân.
3. Chở dê sang bờ đối diện nếu dê cùng bờ với người nông dân.
4. Chở bắp cải sang bờ đối diện nếu bắp cải cùng bờ với người nông dân.

1.3.4 Điều kiện hợp lệ

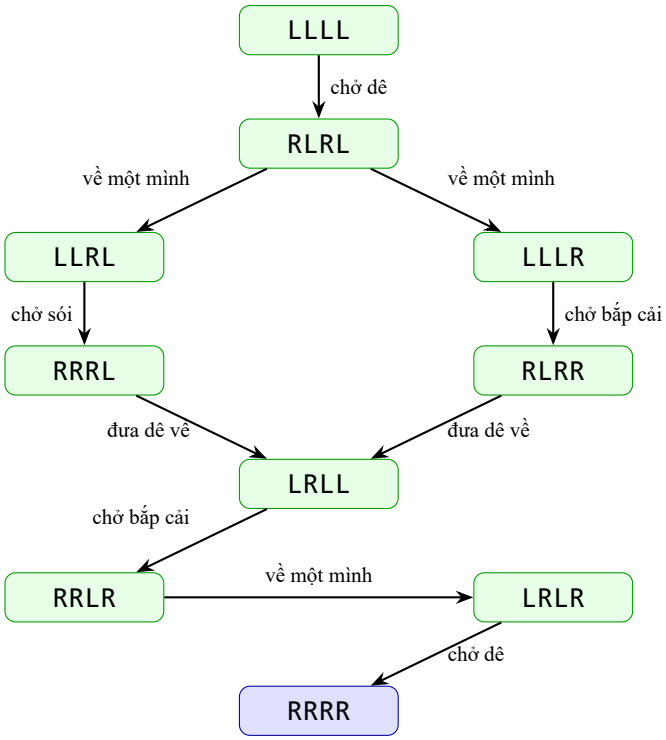
Một trạng thái không hợp lệ nếu xảy ra một trong hai trường hợp:

- Sói và dê ở cùng bờ, còn người nông dân ở bờ bên kia.
- Dê và bắp cải ở cùng bờ, còn người nông dân ở bờ bên kia.

1.3.5 Mục tiêu đồ án

Đồ án hướng đến các mục tiêu chính sau:

1. Mô hình hóa bài toán qua sông thành bài toán tìm kiếm trong không gian trạng thái.
2. Cài đặt và so sánh bốn giải thuật BFS, DFS, A* và IDA*.
3. Xây dựng ứng dụng web trực quan phục vụ việc học tập và trình bày.
4. Giúp người học quan sát được trạng thái, cây tìm kiếm, lời giải và vai trò của heuristic.



Hình 1: Sơ đồ 10 trạng thái hợp lệ của bài toán gốc và các chuyển trạng thái tiêu biểu đến đích

2 Phân tích bài toán

2.1 Đặc trưng của không gian trạng thái

Điểm đáng chú ý của bài toán qua sông là không gian trạng thái nhỏ nhưng bị chi phối mạnh bởi ràng buộc. Điều này có nghĩa là số lượng trạng thái lý thuyết không phản ánh đầy đủ độ khó thật sự của bài toán. Cái khó nằm ở việc nhiều trạng thái sinh ra phải bị loại ngay, và chỉ một số ít nhánh mới có khả năng dẫn đến lời giải hợp lệ.

Ví dụ, từ trạng thái đầu `LLLL`, nếu người nông dân chở sói đi trước thì dê sẽ ở lại với bắp cải mà không có người giám sát. Nếu người nông dân đi một mình, sói lại ở lại với dê. Cả hai đều không hợp lệ. Trên thực tế, hành động đầu tiên an toàn duy nhất là chở dê sang bờ phải. Đây là đặc trưng rất tốt để minh họa cho việc kiểm tra ràng buộc trong AI.

2.2 Biểu diễn trạng thái trong chương trình

Trong dự án, trạng thái được cài đặt bằng kiểu dữ liệu sau:

```
export interface PuzzleState {
  farmer: 'L' | 'R'
  wolf: 'L' | 'R'
  goat: 'L' | 'R'
  cabbage: 'L' | 'R'
```

```
}
```

Listing 1: Khai báo trạng thái trong TypeScript

Mỗi trạng thái sau đó được chuyển thành một khóa chuỗi như `LLLL`, `RLRL`, `RRRR`. Cách biểu diễn này giúp:

- So sánh trạng thái nhanh.
- Dễ lưu vết trạng thái đã gặp bằng `Map`.
- Thuận tiện khi hiển thị trên cây trạng thái của giao diện.
- Dễ giải thích cho người dùng trong quá trình chạy từng bước.

2.3 Vai trò của ràng buộc

Trong các bài toán tìm kiếm đơn giản, đôi khi chỉ cần sinh tất cả các trạng thái con rồi tiếp tục duyệt. Tuy nhiên với bài toán qua sông, bước kiểm tra ràng buộc gần như là một phần của định nghĩa bài toán. Nếu bỏ qua bước này, cây tìm kiếm sẽ có nhiều nhánh vô nghĩa và không còn phản ánh hành vi đúng của bài toán thực tế.

Trong mã nguồn hiện tại, hàm `getInvalidReason` không chỉ trả về việc trạng thái có hợp lệ hay không, mà còn trả về chuỗi mô tả nguyên nhân bị loại. Nhờ đó, giao diện có thể hiển thị trực tiếp cho người dùng biết vì sao một nhánh không được tiếp tục mở rộng. Đây là điểm có giá trị sư phạm rất cao.

2.4 Phân tích heuristic được sử dụng

Để phục vụ A* và IDA*, dự án hiện cài đặt hai heuristic:

2.4.1 Heuristic `remaining-items`

Heuristic này đếm số đối tượng trong tập $\{W, G, C\}$ chưa sang bờ phải:

$$h_1(s) = |\{x \in \{W, G, C\} : x = L\}|$$

Heuristic này đơn giản nhưng trực quan. Nó phản ánh rằng mỗi đối tượng còn ở bờ trái ít nhất vẫn cần được đưa sang bờ phải một lần nữa.

2.4.2 Heuristic `remaining-trips`

Heuristic thứ hai xét thêm tình huống người nông dân đang ở bờ phải nhưng vẫn còn đối tượng ở bờ trái. Khi đó cần ít nhất một chuyến quay về mới có thể tiếp tục vận chuyển:

$$h_2(s) = h_1(s) + \begin{cases} 1 & \text{nếu } F = R \text{ và còn đối tượng ở bờ trái} \\ 0 & \text{ngược lại} \end{cases}$$

Heuristic này gần với chi phí vận chuyển thực tế hơn, vì nó phản ánh đúng tình huống phải quay lại để đón thêm đối tượng.

2.5 Lý do chọn bốn thuật toán

Mặc dù sản phẩm hiện hỗ trợ nhiều thuật toán hơn, báo cáo này chỉ tập trung vào BFS, DFS, A* và IDA* để giữ phạm vi vừa đủ cho môn học nhưng vẫn thể hiện được hai hướng tiếp cận lớn:

- Tìm kiếm mù: BFS, DFS.
- Tìm kiếm có thông tin: A*, IDA*.

Với bốn thuật toán này, người học có thể thấy rõ cách thứ tự duyệt thay đổi khi thuật toán có thêm thông tin từ heuristic.

Bảng 1: Tóm tắt yêu cầu và đặc điểm của bài toán

Khía cạnh	**Mô tả**
Quy mô bài toán	Nhỏ, thích hợp cho trực quan hóa toàn bộ tiến trình
Mức độ ràng buộc	Cao, nhiều trạng thái phải bị loại ngay sau khi sinh
Yêu cầu giải thuật	Cần cho thấy sự khác nhau giữa tìm kiếm mù và có thông tin
Giá trị sư phạm	Dễ trình bày, dễ kiểm chứng, dễ gắn với giao diện minh họa

3 Cơ sở lý thuyết

3.1 BFS – Breadth-First Search

BFS là giải thuật tìm kiếm theo chiều rộng. Thuật toán luôn mở rộng các nút ở độ sâu nhỏ hơn trước, nhờ đó nếu mọi hành động có cùng chi phí thì BFS bảo đảm tìm được lời giải ngắn nhất theo số bước.

Trong bài toán qua sông, mỗi chuyến đi đều được tính là một đơn vị chi phí, do đó BFS rất phù hợp để tìm lời giải tối ưu về số bước.

Ưu điểm:

- Tìm được lời giải tối ưu theo số bước.
- Dễ cài đặt, dễ giải thích và phù hợp cho minh họa trên lớp.
- Là chuẩn so sánh tốt cho các thuật toán khác.

Nhược điểm:

- Tốn bộ nhớ nếu frontier tăng lớn.
- Không tận dụng được tri thức miền bài toán.

Algorithm 1: BFS

Đầu vào: Trạng thái khởi đầu s_0, hàm kiểm tra đích
Đầu ra: Đường đi từ s_0 đến đích hoặc thất bại

```
Khởi tạo hàng đợi với s_0;
while hàng đợi không rỗng do
    Lấy nút đầu tiên trong hàng đợi;
    if nút hiện tại là đích then
        return đường đi
    ;
    Sinh các nút con hợp lệ và thêm vào cuối hàng đợi;
return thất bại;
```

3.2 DFS – Depth-First Search

DFS mở rộng một nhánh theo chiều sâu trước khi quay lui. Thuật toán sử dụng ngăn xếp nên có khuynh hướng đi nhanh vào một nhánh cụ thể. Trên các bài toán nhỏ, DFS có thể tìm lời giải nhanh, nhưng không bảo đảm lời giải tìm được là ngắn nhất.

Ưu điểm:

- Cài đặt đơn giản.
- Thường dùng ít bộ nhớ hơn BFS.
- Thể hiện rõ tư duy thử sâu theo nhánh.

Nhược điểm:

- Không bảo đảm tối ưu.
- Kết quả phụ thuộc mạnh vào thứ tự sinh hành động.

Algorithm 2: DFS

Đầu vào: Trạng thái khởi đầu s_0, hàm kiểm tra đích

Đầu ra: Đường đi hoặc thất bại

```
Khởi tạo ngăn xếp với s_0;
while ngăn xếp không rỗng do
    Lấy nút trên cùng của ngăn xếp;
    if nút hiện tại là đích then
        return đường đi
    ;
    Sinh các nút con hợp lệ và đẩy vào ngăn xếp;
return thất bại;
```

3.3 A* Search

A* là thuật toán tìm kiếm có thông tin, sử dụng hàm đánh giá:

$$f(n) = g(n) + h(n)$$

Trong đó $g(n)$ là chi phí thực từ trạng thái đầu đến nút hiện tại, còn $h(n)$ là ước lượng chi phí còn lại đến đích. A* ưu tiên những nút vừa có chi phí đi đến thấp, vừa hứa hẹn ở gần mục tiêu.

Trong đồ án, A* sử dụng frontier sắp theo giá trị f, nếu bằng nhau thì xét thêm h và g. Điều này đủ để thể hiện cơ chế hoạt động của thuật toán trong phạm vi một bài toán nhỏ.

Algorithm 3: A* Search

Đầu vào: Trạng thái khởi đầu s_0, hàm kiểm tra đích, heuristic h

Đầu ra: Đường đi tối ưu hoặc thất bại

```
Khởi tạo biên ưu tiên theo f(n) = g(n) + h(n);
while biên không rỗng do
    Lấy nút có giá trị f nhỏ nhất;
    if nút hiện tại là đích then
        return đường đi
    ;
    Sinh các nút con hợp lệ và cập nhật vào biên;
return thất bại;
```

3.4 IDA* – Iterative Deepening A*

IDA* là biến thể của A* theo hướng tiết kiệm bộ nhớ. Thay vì lưu cả frontier lớn, IDA* duyệt sâu theo ngưỡng của hàm f. Khi gặp nút có giá trị f lớn hơn ngưỡng hiện tại, thuật toán tạm dừng mở rộng nút đó và ghi nhận giá trị vượt ngưỡng nhỏ nhất cho vòng lặp kế tiếp.

Ưu điểm:

- Tận dụng được heuristic như A*.
- Dùng ít bộ nhớ hơn A*.

Nhược điểm:

- Phải lặp nhiều vòng với các ngưỡng tăng dần.
- Có thể tốn thêm thời gian do phải duyệt lại.

Algorithm 4: IDA*

Đầu vào: Trạng thái khởi đầu s_0, hàm kiểm tra đích, heuristic h

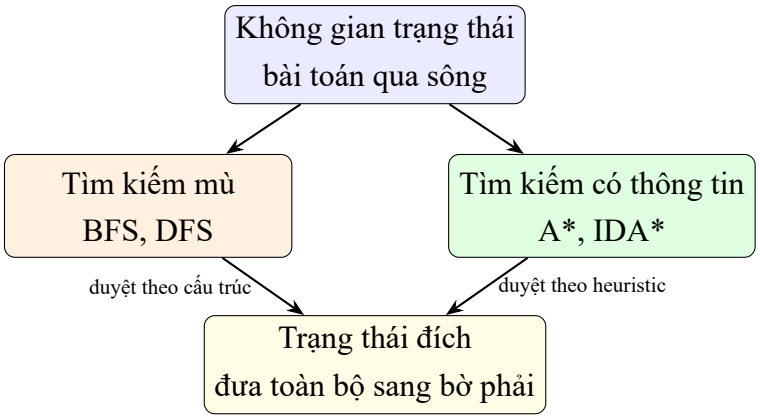
Đầu ra: Đường đi tối ưu hoặc thất bại

```
bound ← h(s_0);
while true do
    Chạy DFS với điều kiện f(n) ≤ bound;
    if tìm thấy đích then
        return đường đi
    ;
    bound ← giá trị f vượt ngưỡng nhỏ nhất tiếp theo;
```

3.5 So sánh lý thuyết giữa bốn thuật toán

Bảng 2: So sánh lý thuyết giữa bốn thuật toán

Thuật toán	Loại tìm kiếm	Cấu trúc chính	Tối ưu	Bộ nhớ
BFS	Tìm kiếm mù	Hàng đợi	Có, khi chi phí đều nhau	Cao
DFS	Tìm kiếm mù	Ngăn xếp	Không bảo đảm	Thấp
A*	Tìm kiếm có thông tin	Biên theo $f = g + h$	Có nếu heuristic phù hợp	Cao
IDA*	Tìm kiếm có thông tin	DFS theo ngưỡng f	Có nếu heuristic phù hợp	Thấp hơn A*



Hình 2: Hai nhóm thuật toán chính được so sánh trong báo cáo

4 Thiết kế và cài đặt

4.1 Công nghệ sử dụng

Nhóm xây dựng sản phẩm dưới dạng ứng dụng web trực quan. Đây là lựa chọn phù hợp vì giúp việc trình bày đồ án sinh động hơn nhiều so với một chương trình console thuần văn bản. Người dùng có thể thay đổi thuật toán, chọn heuristic, xem trạng thái hiện tại, theo dõi cây tìm kiếm và so sánh kết quả giữa các lần chạy.

Các công nghệ chính gồm:

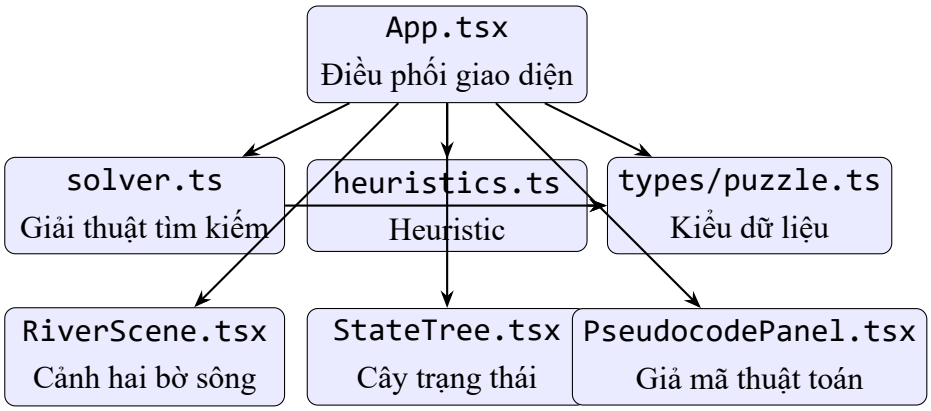
- React 19 để xây dựng giao diện theo thành phần.
- TypeScript để đảm bảo chặt chẽ về kiểu dữ liệu.
- Vite cho môi trường phát triển và đóng gói nhanh.
- Ant Design để cung cấp hệ thống thành phần giao diện.
- React Flow để biểu diễn cây trạng thái.
- SVG để minh họa người nông dân, sói, dê, bắp cải và thuyền.

4.2 Kiến trúc chương trình

Mã nguồn được chia thành các nhóm chức năng rõ ràng:

- `src/algorithms/solver.ts`: cài đặt các giải thuật tìm kiếm, sinh hành động, kiểm tra trạng thái và tính toán chỉ số.
- `src/algorithms/heuristics.ts`: chứa các heuristic dùng cho A* và IDA*.
- `src/types/puzzle.ts`: mô tả kiểu dữ liệu cho trạng thái, kết quả, nút tìm kiếm.

- `src/components/RiverScene.tsx`: hiển thị trực quan trạng thái ở hai bờ sông.
- `src/components/StateTree.tsx`: dựng cây tìm kiếm bằng React Flow.
- `src/components/PseudocodePanel.tsx`: hiển thị giả mã của thuật toán đang chọn.
- `src/App.tsx`: điều phối toàn bộ giao diện và logic tương tác.



Hình 3: Sơ đồ kiến trúc chính của chương trình

4.3 Cấu trúc dữ liệu và cách lưu vết tiến trình

Ngoài trạng thái, mỗi nút trong cây tìm kiếm còn lưu thêm các trường `parentId`, `depth`, `g`, `h`, `f`, `valid`, `prunedReason` và `expandedOrder`. Điều này cho phép hệ thống vừa thực thi thuật toán vừa ghi lại toàn bộ tiến trình để phục vụ cho giao diện.

Nhờ cách thiết kế này, ứng dụng có thể:

- Hiển thị cây tìm kiếm đầy đủ.
- Tô sáng đường đi lời giải.
- Chỉ ra các nút bị loại vì vi phạm ràng buộc.
- Phát lại quá trình mở rộng từng bước.

4.4 Kiểm tra ràng buộc trong mã nguồn

Phần cốt lõi của bài toán là loại bỏ các trạng thái không an toàn. Điều này được cài đặt trực tiếp trong mã nguồn như sau:

```
if (state.wolf === state.goat && state.farmer !== state.goat) {
  return 'Sói ẽs ăn dê khi nông dân ắvng ặmt.'
}

```

```
if (state.goat === state.cabbage && state.farmer !== state.goat) {
  return 'Dê ẽs ăn ắbp ảci khi nông dân ắvng ặmt.'
}
```

Listing 2: Kiểm tra ràng buộc của bài toán

Điểm hay của cài đặt này là chương trình không chỉ loại bỏ trạng thái sai, mà còn lưu lý do bị loại để hiển thị trực tiếp lên giao diện. Nhờ đó, người học không chỉ biết trạng thái đó sai mà còn hiểu sai ở đâu.

4.5 Cài đặt từng nhóm chức năng

4.5.1 BFS và DFS

Hai thuật toán này dùng chung phần sinh trạng thái, chỉ khác nhau ở cách lấy phần tử từ frontier. Với BFS, chương trình lấy phần tử đầu danh sách; với DFS, chương trình lấy phần tử cuối danh sách. Cách cài đặt này giúp việc đối chiếu hành vi giữa hai thuật toán trở nên rõ ràng và dễ kiểm soát.

4.5.2 A*

A* sử dụng giá trị $f = g + h$ để sắp xếp frontier. Đây là điểm thể hiện rõ tri thức miền bài toán được đưa vào tiến trình tìm kiếm. Các trạng thái có tiềm năng hơn sẽ được ưu tiên mở rộng trước.

4.5.3 IDA*

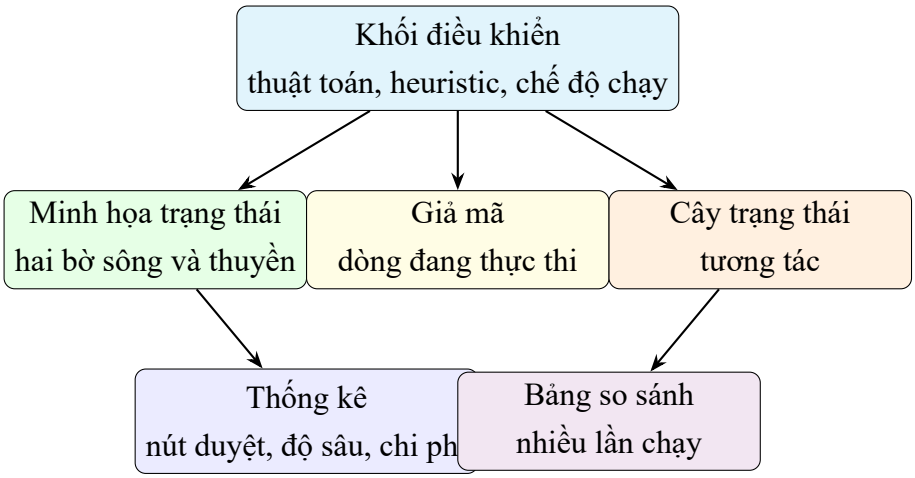
IDA* được cài đặt qua các vòng lặp tăng ngưỡng. Trong mỗi vòng, chương trình dùng DFS nhưng chỉ xét các nút có giá trị f không vượt quá `threshold`. Cách tiếp cận này vừa tiết kiệm bộ nhớ, vừa giữ được định hướng của heuristic.

4.6 Thiết kế giao diện trực quan

Giao diện được tổ chức để phục vụ cả mục đích chạy thử lẫn mục đích giảng dạy. Các thành phần chính gồm:

- Khối chọn thuật toán, heuristic và chế độ chạy.
- Vùng minh họa người nông dân, sói, dê, bắp cải ở hai bờ sông.
- Khung giả mã với dòng được tô sáng theo bước hiện tại.
- Cây trạng thái cho phép chọn từng nút để phân tích.

- Thẻ thống kê hiển thị số nút duyệt, số nút sinh, biên tối đa, độ sâu lời giải, chi phí lời giải và thời gian.
- Bảng lịch sử dùng để so sánh các lần chạy liên tiếp.



Hình 4: Bố cục chức năng chính của giao diện web trực quan

5 Kết quả và đánh giá

5.1 Kiểm tra khả năng hoạt động của hệ thống

Dự án đã được kiểm tra bằng lệnh build:

```
npm run build
```

Listing 3: Lệnh build dự án web

Kết quả build thành công cho thấy mã nguồn của sản phẩm đang ở trạng thái có thể sử dụng. Đây là điều kiện cần để báo cáo có giá trị thực tế, không chỉ dừng ở mức mô tả lý thuyết.

5.2 Lời giải chuẩn của bài toán

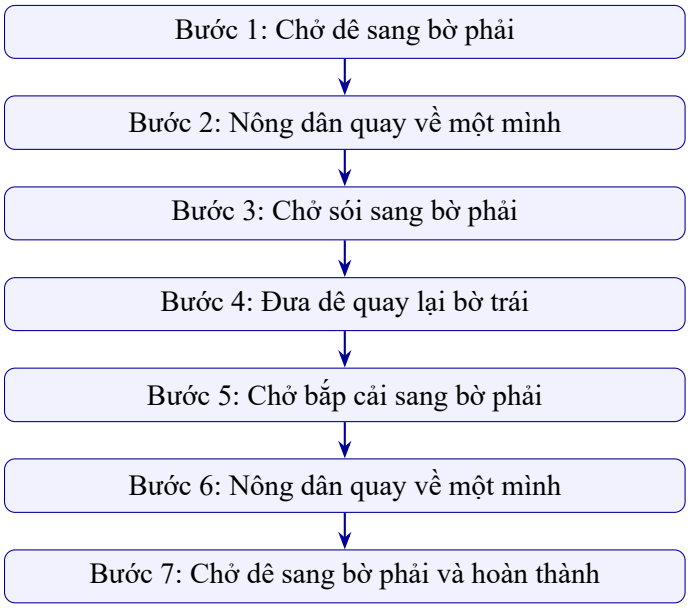
Một lời giải hợp lệ và tối ưu của bài toán gồm 7 bước:

1. Nông dân chở dê sang bờ phải.
2. Nông dân đi một mình về bờ trái.
3. Nông dân chở sói sang bờ phải.
4. Nông dân chở dê về bờ trái.
5. Nông dân chở bắp cải sang bờ phải.

6. Nông dân đi một mình về bờ trái.

7. Nông dân chở dê sang bờ phải.

Lời giải này được dùng làm chuẩn để kiểm tra tính đúng đắn của các thuật toán trong chương trình.



Hình 5: Minh họa chuỗi hành động của lời giải tối ưu 7 bước

5.3 Đánh giá theo từng thuật toán

Bảng 3: Đánh giá tổng quan trên bài toán gốc

Thuật toán	**Kết quả**	**Nhận xét**
BFS	Tìm được lời giải 7 bước	Ổn định, dễ giải thích, phù hợp để minh họa tìm kiếm theo tầng và làm chuẩn so sánh
DFS	Tìm được lời giải nhưng không bảo đảm tối ưu tổng quát	Đơn giản, ít tốn bộ nhớ hơn BFS nhưng phụ thuộc vào thứ tự sinh nhánh
A*	Tìm được lời giải với định hướng từ heuristic	Thể hiện rõ tác dụng của heuristic trong việc ưu tiên các trạng thái hứa hẹn
IDA*	Tìm được lời giải với heuristic và cơ chế ngưỡng	Tiết kiệm bộ nhớ hơn A*, nhưng phải lặp nhiều vòng theo các ngưỡng tăng dần

5.4 So sánh tìm kiếm mù và tìm kiếm có thông tin

Từ góc độ học thuật, giá trị lớn nhất của đồ án nằm ở chỗ nó cho phép quan sát trực tiếp sự khác nhau giữa hai cách tiếp cận:

- BFS và DFS là tìm kiếm mù, nghĩa là không biết trạng thái nào gần đích hơn. Chúng chỉ quyết định dựa trên cấu trúc frontier.
- A* và IDA* là tìm kiếm có thông tin, nghĩa là có thêm heuristic để đánh giá mức độ hứa hẹn của từng trạng thái.

Sự khác biệt này không chỉ nằm trên công thức mà còn thể hiện bằng hành vi duyệt cây khác nhau. Khi xem trên giao diện, người dùng có thể nhận ra ngay rằng A* và IDA* thường tập trung sớm hơn vào các trạng thái có lợi cho mục tiêu.

5.5 Giá trị của giao diện trực quan

Nếu chỉ cài đặt giải thuật dưới dạng chương trình dòng lệnh, người dùng sẽ khó hình dung tại sao một nhánh bị loại hoặc vì sao một thuật toán mở rộng nút khác với thuật toán còn lại. Với giao diện hiện tại, nhóm đã giải quyết khá tốt vấn đề này:

- Cảnh hai bờ sông cho phép xem trạng thái ngay lập tức.
- Cây trạng thái cho thấy quan hệ cha con giữa các nút.
- Giả mã giúp liên hệ lý thuyết với hành vi thực thi.
- Bảng thống kê hỗ trợ so sánh nhanh các lần chạy.

Đây là yếu tố làm cho đồ án có chất lượng trình bày tốt hơn đáng kể so với một bản cài đặt thuần túy.

5.6 Ưu điểm và hạn chế

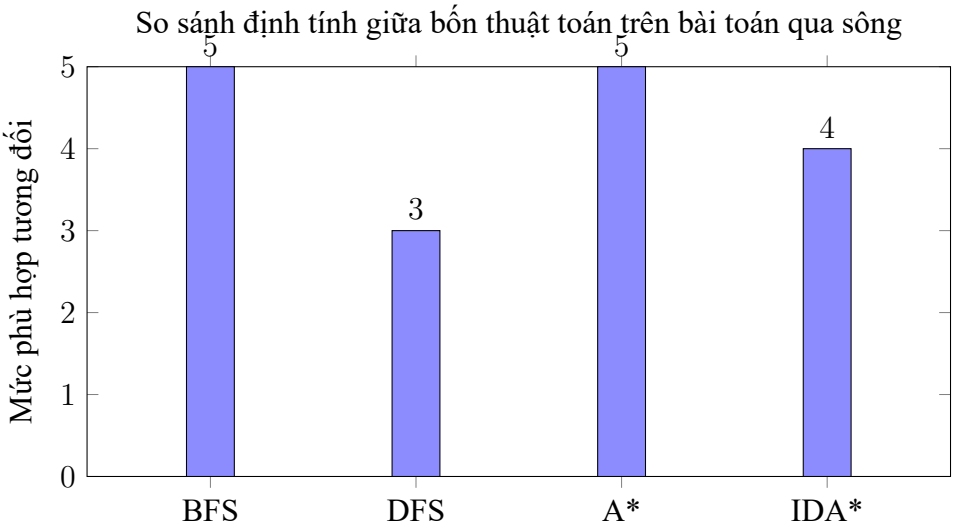
Ưu điểm:

- Cài đặt đầy đủ bốn thuật toán tiêu biểu của học phần.
- Tổ chức mã nguồn rõ ràng, tách biệt giữa thuật toán và giao diện.
- Giao diện đẹp, có khả năng minh họa tốt.
- Có thể dùng trực tiếp để báo cáo trên lớp.

Hạn chế:

- Bài toán còn nhỏ nên chưa thể hiện áp lực hiệu năng lớn.

- Chưa xuất tự động các số liệu thực nghiệm ra báo cáo riêng.
- Chưa có ảnh chụp giao diện thật được nhúng trực tiếp vào báo cáo.



Hình 6: Biểu đồ định tính về mức phù hợp của các thuật toán cho mục tiêu minh họa bài toán

PHẦN KẾT LUẬN

1. Kết luận

Đồ án đã hoàn thành các mục tiêu trọng tâm của môn học Nhập môn Trí tuệ nhân tạo CT223H. Nhóm đã mô hình hóa bài toán qua sông dưới dạng bài toán tìm kiếm, cài đặt bốn thuật toán BFS, DFS, A* và IDA*, đồng thời xây dựng một ứng dụng web trực quan giúp người học quan sát được trạng thái, cây tìm kiếm, lời giải và vai trò của heuristic.

Qua đồ án có thể rút ra các kết luận chính:

- BFS là lựa chọn tốt khi cần lời giải tối ưu theo số bước.
- DFS đơn giản nhưng không phải lúc nào cũng là lựa chọn tối ưu.
- A* cho thấy rõ sức mạnh của heuristic trong việc định hướng tìm kiếm.
- IDA* là phương án phù hợp khi muốn giữ lợi ích của heuristic nhưng giảm chi phí bộ nhớ.

Điểm nổi bật của đồ án là đã chuyển phần lý thuyết thành một sản phẩm chạy được, dễ trình bày, dễ quan sát và có giá trị học tập thực tế.

2. Hướng phát triển

Trong các phiên bản tiếp theo, nhóm có thể mở rộng theo các hướng sau:

- Bổ sung ảnh chụp giao diện thật vào báo cáo để tăng tính trực quan.
- Mở rộng bài toán với nhiều đối tượng hơn hoặc thay đổi sức chứa của thuyền.
- Bổ sung chức năng xuất số liệu thực nghiệm tự động ra CSV hoặc PDF.
- Tích hợp giải thích heuristic chi tiết theo từng trạng thái.

TÀI LIỆU THAM KHẢO

1. Stuart Russell, Peter Norvig. *Artificial Intelligence: A Modern Approach*. Pearson.
2. Tài liệu đề tài môn học CT223H: bài toán qua sông người nông dân, sói, dê và bắp cải.
3. Mã nguồn đồ án trong thư mục `src` của dự án ứng dụng web trực quan.
4. Tài liệu chính thức của React. https://react.dev
5. Tài liệu chính thức của Ant Design. https://ant.design
6. Tài liệu chính thức của React Flow. https://reactflow.dev